

Comparación experimental de GA, ACO, PSO, ascenso de colinas y temple simulado en el problema del viajante

Taller 4 – Inteligencia Artificial

Jimmy Millán, Santiago León, Natalia Orjuela

Repositorio: <https://github.com/Nataorjuela/taller4.git>

Resumen

Se comparan cinco metaheurísticas sobre el TSP euclidiano con $n \in \{20, 50, 100\}$ ciudades (5 instancias por tamaño, 30 ejecuciones independientes por combinación), bajo un presupuesto idéntico de evaluaciones de la función objetivo (10 000, 30 000 y 60 000 FEs) y semillas emparejadas. ACO obtuvo las rutas más cortas y más estables en todos los tamaños (error mediano 0,00/0,66/2,66 %); SA fue segundo para $n \geq 50$; HC fue el más rápido (0,06 s con $n = 100$, ≈ 90 veces menos que ACO); PSO con claves aleatorias fue el peor por amplio margen. La prueba de Friedman rechazó la igualdad en los tres tamaños ($p < 10^{-16}$), con un tamaño del efecto creciente ($W = 0,63 \rightarrow 0,92$). No existe un ganador absoluto: la mejor calidad (ACO) no coincide con el menor tiempo (HC) ni con la menor memoria medida en los tamaños grandes (GA).

1. Metodología

Problema. Dadas n ciudades con coordenadas uniformes en $[0, 100]^2$, se minimiza $f(\pi) = \sum_{k=1}^{n-1} d_{\pi_k \pi_{k+1}} + d_{\pi_n \pi_1}$ con d_{ij} euclidiana y π una permutación. Todas las implementaciones usan la misma matriz de distancias y la misma función objetivo, centralizadas en un objeto **Evaluador** que cuenta las FEs, conserva la mejor solución histórica y registra el historial (FEs, mejor costo). Ningún algoritmo puede exceder el presupuesto.

Algoritmos y codificación (Tabla 1). HC y SA comparten el mismo generador de vecinos 2-opt; el cambio de costo se calcula en $O(1)$ con $\Delta = d_{ac} + d_{bd} - d_{ab} - d_{cd}$ y cada vecino evaluado cuenta como 1 FE (se verificó que Δ coincide con recalcular f).

Alg.	Representación	Operadores	Parámetros (fase piloto)
GA	permutación	torneo, cruce OX, mutación por inversión, elitismo	$P = 50$, $k = 3$, $p_c = 0,9$, $p_m = 0,3$, élite 2
ACO	ruta construida	Ant System elitista: cada paso prefiere caminos con más señal acumulada y ciudades cercanas; luego evapora señal vieja y refuerza las mejores rutas	$m = n$, $\alpha = 1$, $\beta = 2$, $\rho = 0,3$, $e = 5$
PSO	claves aleatorias, ruta = argsort($\mathbf{x}$)	$\mathbf{v} \leftarrow w\mathbf{v} + c_1\mathbf{r}_1(\mathbf{p} - \mathbf{x}) + c_2\mathbf{r}_2(\mathbf{g} - \mathbf{x})$	$S = 50$, $w = 0,729$, $c_{1,2} = 1,494$, $v_{\max} = 0,5$
HC	permutación	vecino 2-opt aleatorio; se acepta si mejora; reinicio tras paciencia	paciencia = $2 \cdot n(n-1)/2$
SA	permutación	mismo 2-opt; Metropolis $e^{-\Delta/T}$; $T_0 = -\Delta/\ln p_0$; $T_{k+1} = \alpha T_k$, $\alpha = r^{1/K}$	$p_0 = 0,2$, $r = 0,01$

Tabla 1: Algoritmos, codificación y parámetros. Los parámetros se fijaron en una fase piloto con instancias separadas (semillas 901–903, $n = 50$).

Protocolo. Instancias con semillas públicas 101–105; 30 repeticiones por algoritmo, tamaño e instancia (2250 corridas; 450 adicionales para el híbrido). La semilla de cada corrida depende de (2026, n , instancia, repetición) y es la misma para todos los algoritmos, lo que permite pruebas emparejadas. El tiempo se mide con **perf_counter** solo alrededor del algoritmo; la memoria pico con **tracemalloc** en una segunda pasada con la misma semilla (repeticiones 0–4), porque **tracemalloc** ralentiza el código de forma desigual entre algoritmos. Todo se ejecutó en el mismo equipo (2 procesos). La configuración completa está en **configuracion.json**.

Métricas. f^* = mejor valor encontrado por cualquier algoritmo en todas las corridas de la instancia. $\text{Error}_r = 100(f_r - f^*)/f^*$; éxito = % de corridas con error $\leq 1\%$ ($R = 30$ por instancia); mejora = $100(f_{\text{inicial}} - f_r)/f_{\text{inicial}}$. Las curvas de convergencia se evalúan en 50 puntos comunes de FEs antes de promediar. Se aplica Friedman por tamaño (150 bloques) con W de Kendall, y post hoc de Wilcoxon pareado con corrección de Holm y correlación biserial de rangos.

2. Resultados

2.1. Eficiencia computacional y escalabilidad

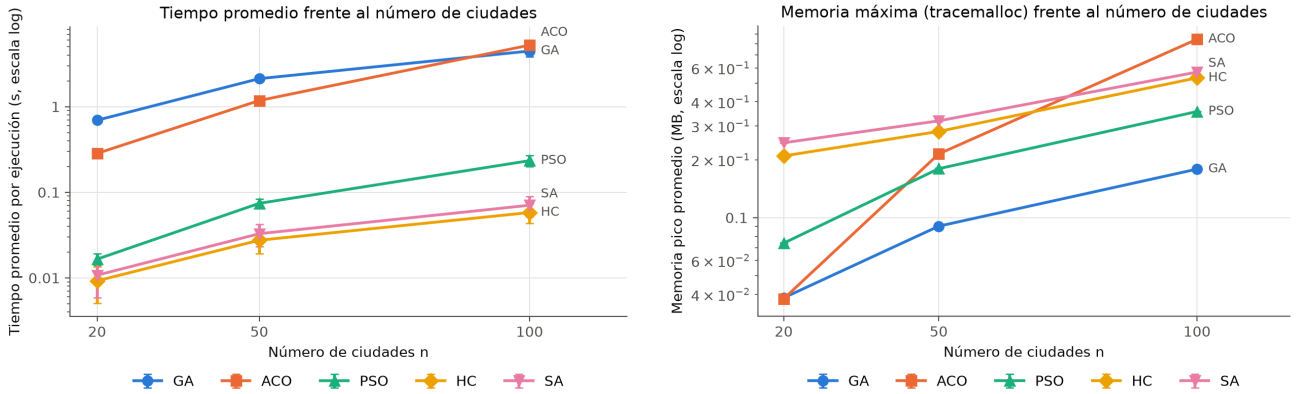


Figura 1: Tiempo promedio por corrida y memoria pico (tracemalloc) frente a n (escala log; barras: ± 1 desviación).

n	Alg.	tiempo (s)	ms/1000 FEs	memoria (MB)	éxito (%)	FEs al 1 % (med.)	llegaron
20	GA	0,694	69,4	0,038	36,0	3 488	54/150
	ACO	0,284	28,4	0,038	99,3	242	149/150
	PSO	0,017	1,7	0,074	0,7	3 417	1/150
	HC	0,009	0,9	0,209	73,3	3 076	110/150
	SA	0,011	1,1	0,245	70,7	3 324	106/150
50	GA	2,124	70,8	0,090	2,0	22 710	3/150
	ACO	1,179	39,3	0,214	66,0	7 713	99/150
	PSO	0,074	2,5	0,180	0,0	—	0/150
	HC	0,028	0,9	0,280	2,7	11 162	4/150
	SA	0,033	1,1	0,318	16,0	18 560	24/150
100	GA	4,463	74,4	0,179	0,0	—	0/150
	ACO	5,228	87,1	0,845	17,3	34 784	26/150
	PSO	0,234	3,9	0,356	0,0	—	0/150
	HC	0,058	1,0	0,532	0,0	—	0/150
	SA	0,071	1,2	0,570	4,0	53 696	6/150

Tabla 2: Tiempo, memoria, tasa de éxito y FEs necesarias para error $\leq 1\%$ (mediana entre las corridas que llegaron; “llegaron” de 150).

HC y SA son los más rápidos (0,06–0,07 s con $n = 100$) y su costo por FE es casi constante (≈ 1 ms/1000 FEs), gracias a la evaluación incremental $O(1)$. ACO escala peor en tiempo (28→87 ms/1000 FEs) porque construir una ruta cuesta $O(n^2)$, y en memoria ($\times 22$ de $n = 20$ a $n = 100$) por sus matrices $n \times n$. GA usa la menor memoria medida para $n = 50$ y $n = 100$ (con empate práctico entre GA y ACO en $n = 20$), pero su costo por FE (≈ 70 ms/1000) está dominado por operaciones en Python. En calidad, ACO es quien mejor escala: es el único que alcanza el umbral del 1 % de forma recurrente con $n = 100$ (26/150 corridas; SA 6/150). Rapidez y calidad no coinciden: HC termina 90 veces antes que ACO pero con el triple de error.

2.2. Convergencia, calidad y estabilidad

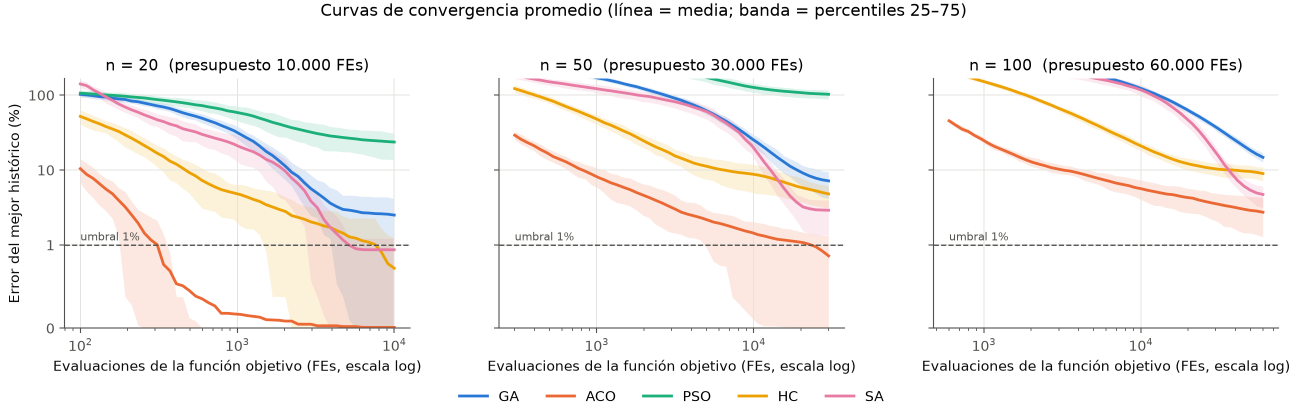


Figura 2: Error del mejor histórico frente a FEs (media y percentiles 25-75 de 150 corridas; ejes logarítmicos, eje y lineal en $[0, 1]$).

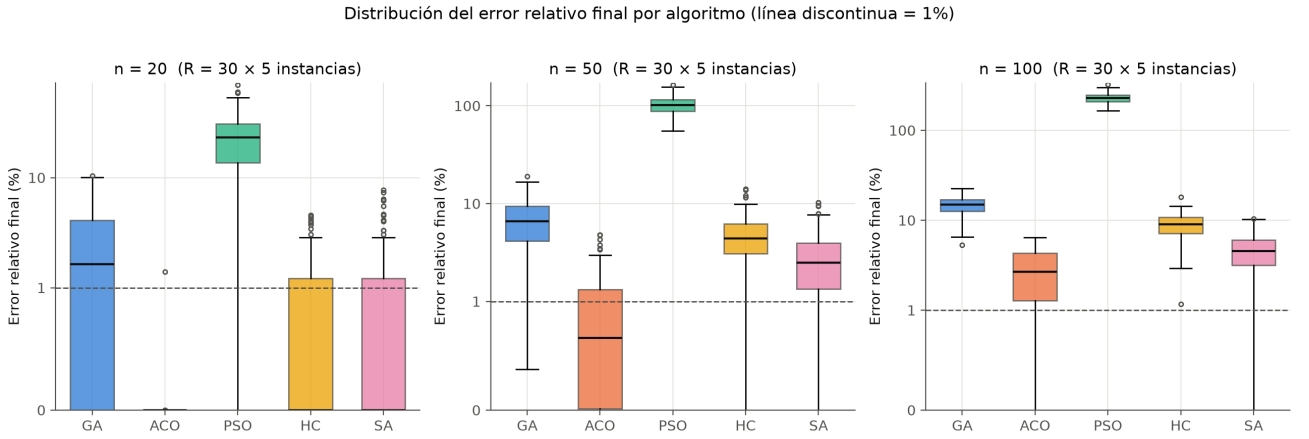


Figura 3: Diagramas de caja del error relativo final (150 corridas por algoritmo y tamaño).

n	Alg.	mejor	media	mediana	desv. [†]	RIC	éxito (%)	mejora (%)
20	GA	0,00	2,51	1,63	2,14	4,12	36,0	61,7
	ACO	0,00	0,01	0,00	0,05	0,00	99,3	40,9
	PSO	0,00	23,67	23,10	13,07	17,30	0,7	53,8
	HC	0,00	0,72	0,00	0,89	1,22	73,3	62,3
	SA	0,00	0,94	0,00	0,98	1,22	70,7	62,2
50	GA	0,37	7,17	6,61	3,70	5,31	2,0	76,3
	ACO	0,00	0,87	0,66	0,62	1,31	66,0	55,8
	PSO	55,13	102,71	101,72	19,69	27,99	0,0	55,5
	HC	0,00	4,80	4,41	2,36	3,09	2,7	76,8
	SA	0,00	2,91	2,50	2,06	2,61	16,0	77,2
100	GA	5,29	14,78	14,91	3,30	4,22	0,0	82,2
	ACO	0,00	2,73	2,66	0,89	3,04	17,3	63,5
	PSO	165,26	230,27	228,01	24,07	39,56	0,0	48,6
	HC	1,17	8,98	8,97	2,52	3,72	0,0	83,1
	SA	0,00	4,72	4,56	2,19	2,87	4,0	83,8

Tabla 3: Error relativo final (%). [†]Desviación estándar dentro de cada instancia, promediada. El costo absoluto (mejor, media, mediana y desviación) por instancia está en `resultados/tabla_rendimiento_por_instancia.csv`.

ACO arranca con rutas buenas gracias a la visibilidad $1/d$ y mantiene la ventaja durante todo el presupuesto (Fig. 2). HC desciende rápido y se estanca; SA muestra la forma en “S” típica: explora con temperatura alta y

cae al enfriarse, superando a HC para $n \geq 50$. GA aún mejora al final del presupuesto (mejor solución en la FE mediana del 98 % con $n = 100$); PSO se estanca debido a la falta de localidad de las claves aleatorias. La Fig. 4 muestra las rutas de la corrida mediana: ACO, SA y HC no presentan cruces; PSO está lleno de ellos.

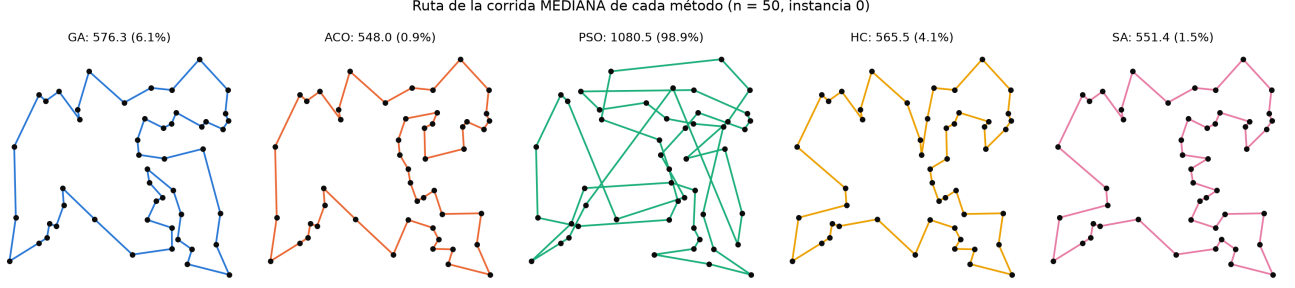


Figura 4: Ruta de la corrida mediana de cada método en la instancia 0 con $n = 50$ (entre paréntesis, error respecto a f^*).

2.3. Análisis estadístico y ranking

n	χ_F^2	p	W	$\bar{R}$ GA	$\bar{R}$ ACO	$\bar{R}$ PSO	$\bar{R}$ HC	$\bar{R}$ SA
20	379,6	$< 10^{-16}$	0,633	3,31	1,84	4,95	2,31	2,58
50	467,4	$< 10^{-16}$	0,779	3,50	1,31	5,00	3,02	2,17
100	549,5	$< 10^{-16}$	0,916	3,93	1,26	5,00	2,93	1,88

Tabla 4: Friedman ($N = 150$, $k = 5$) y rangos promedio (1 = mejor).

Par	$n = 20$	$n = 50$	$n = 100$
GA vs ACO	ACO (0,88; $< 10^{-4}$)	ACO (0,99; $< 10^{-4}$)	ACO (1,00; $< 10^{-4}$)
GA vs PSO	GA ($-0,99$; $< 10^{-4}$)	GA ($-1,00$; $< 10^{-4}$)	GA ($-1,00$; $< 10^{-4}$)
GA vs HC	HC (0,70; $< 10^{-4}$)	HC (0,54; $< 10^{-4}$)	HC (0,96; $< 10^{-4}$)
GA vs SA	SA (0,63; $< 10^{-4}$)	SA (0,85; $< 10^{-4}$)	SA (1,00; $< 10^{-4}$)
ACO vs PSO	ACO ($-1,00$; $< 10^{-4}$)	ACO ($-1,00$; $< 10^{-4}$)	ACO ($-1,00$; $< 10^{-4}$)
ACO vs HC	ACO ($-0,51$; $< 10^{-4}$)	ACO ($-0,96$; $< 10^{-4}$)	ACO ($-1,00$; $< 10^{-4}$)
ACO vs SA	ACO ($-0,63$; $< 10^{-4}$)	ACO ($-0,77$; $< 10^{-4}$)	ACO ($-0,66$; $< 10^{-4}$)
PSO vs HC	HC (1,00; $< 10^{-4}$)	HC (1,00; $< 10^{-4}$)	HC (1,00; $< 10^{-4}$)
PSO vs SA	SA (1,00; $< 10^{-4}$)	SA (1,00; $< 10^{-4}$)	SA (1,00; $< 10^{-4}$)
HC vs SA	HC ($-0,21$; 0,0258)	SA (0,65; $< 10^{-4}$)	SA (0,92; $< 10^{-4}$)

Tabla 5: Wilcoxon pareado con Holm: ganador (biserial de rangos; p ajustado). Todas las diferencias son significativas a $\alpha = 0,05$.

n	Alg.	calidad	estab.	veloc.	global	$\bar{R}$	desv. error	AUC
20	ACO	1	1	1	1	1,84	0,05	2,5
	HC	2	2	2	2	2,31	0,89	22,9
	SA	3	3	3	3	2,58	0,98	64,2
	GA	4	4	4	4	3,31	2,14	79,1
	PSO	5	5	5	5	4,95	13,07	120,2
50	ACO	1	1	1	1	1,31	0,62	12,6
	SA	2	2	3	2	2,17	2,06	157,7
	HC	3	3	2	3	3,02	2,36	65,9
	GA	4	4	4	4	3,50	3,70	208,0
	PSO	5	5	5	5	5,00	19,69	389,0
100	ACO	1	1	1	1	1,26	0,89	22,1
	SA	2	2	3	2	1,88	2,19	284,9
	HC	3	3	2	3	2,93	2,52	120,9
	GA	4	4	4	4	3,93	3,30	363,7
	PSO	5	5	5	5	5,00	24,07	726,5

Tabla 6: Ranking por calidad (rango de Friedman), estabilidad (desviación del error dentro de instancia) y velocidad (área bajo la curva de error en log FEs).

Las diferencias son significativas en todos los pares y tamaños. El tamaño del efecto crece con n : W pasa de 0,63 a 0,92 y la ventaja de ACO sobre HC de biserial 0,51 a 1,00. La única diferencia pequeña es HC vs. SA con $n = 20$ (biserial $-0,21$, $p = 0,026$), que se invierte con fuerza a favor de SA para $n = 50$ y $n = 100$.

3. Discusión

1. **Menor costo y mayor estabilidad:** ACO en los tres tamaños (Tablas 3 y 6).
2. **Buenas soluciones con menos FEs:** ACO (mediana de 242 FEs para error $\leq 1\%$ con $n = 20$; 7713 con $n = 50$). HC es el siguiente en velocidad de convergencia.
3. **Menor tiempo:** HC; **menor memoria medida:** GA en los tamaños grandes (con empate práctico frente a ACO en $n = 20$). No coinciden con el de mejor calidad: ACO es el más lento y el de mayor memoria solo con $n = 100$.
4. **Cambio del ranking con n :** extremos estables (ACO 1.º, PSO 5.º, GA 4.º); HC y SA se intercambian: con 20 ciudades los reinicios de HC bastan, con 50 y 100 la aceptación de empeoramientos de SA es decisiva.
5. **Población vs. trayectoria:** la población ayuda cuando comparte información útil (feromona + heurística en ACO); en GA, con presupuesto fijo, cada generación consume $P - 2$ FEs y OX rompe aristas buenas, así que converge lento; en PSO la codificación impide aprovechar la memoria personal y social. La trayectoria única es barata por FE y explota bien la vecindad 2-opt, pero se atasca en óptimos locales.
6. **Aceptar empeoramientos / diversidad:** útil cuando el paisaje tiene muchos óptimos locales y hay presupuesto para enfriar: SA reduce a la mitad el error de HC con $n = 50$ y $n = 100$, pero con 10 % del presupuesto va 4–5 veces peor. Demasiada diversidad es dañina (piloto: GA con $p_m = 0,6$ y $P = 100$ fue 7 veces peor).
7. **No hay ganador absoluto** (coherente con el teorema *No Free Lunch*). Respuesta rápida: HC con 2-opt; alta calidad: ACO; menor memoria medida en este experimento: GA. Si se busca una implementación de trayectoria con poca memoria teórica, SA puede usar una sola ruta y calcular distancias al vuelo.

Amenaza a la validez. f^* es el mejor valor encontrado y no el óptimo: los errores son cotas inferiores y favorecen al algoritmo que define la referencia (ACO con $n = 100$). Además, los parámetros se ajustaron solo con $n = 50$ y los tiempos mezclan algoritmo e implementación (grado de vectorización en NumPy). **Mejora propuesta:** usar instancias con óptimo conocido (TSPLIB o Concorde), ajuste automático de parámetros por tamaño (irace) e implementaciones compiladas para comparar tiempos.

Híbrido ACO + 2-opt (opcional). La mejor hormiga de cada iteración se mejora con $5n$ propuestas 2-opt (mismas FEs totales). El híbrido supera a HC (92–96 % de corridas para $n \geq 50$) y a SA (65–74 %) con $p < 10^{-4}$, pero no a ACO puro: empata con $n = 20$ ($p = 0,31$) y pierde con $n = 50$ y $n = 100$ (error mediano 1,03 vs. 0,66 y 3,59 vs. 2,66 %; $p < 0,01$). La búsqueda local consume $5n$ FEs por iteración y reduce las iteraciones de la colonia de 600 a ≈ 100 ; es 5 veces más rápido en tiempo, así que con un criterio de parada por tiempo el balance podría invertirse.

4. Conclusiones

Una comparación justa exige controlar el problema, el presupuesto (FEs) y las semillas, y analizar a la vez calidad, costo y variabilidad. Con ese control, ACO fue el método de mejor calidad y estabilidad, HC y SA los de menor costo computacional (con SA claramente mejor que HC en calidad para $n \geq 50$), GA quedó en un punto intermedio que mejoraría con más presupuesto, y PSO con claves aleatorias resultó inadecuado para el TSP. Las conclusiones se apoyan en 150 corridas emparejadas por tamaño y en pruebas no paramétricas con corrección por comparaciones múltiples, no solo en promedios.